

Layer 01: Staging (Bronze)

Script: `sql/01_staging.sql` | **Status:** Complete | **Next layer:** Profiling (`02_profiling.sql`)

1. Purpose

Land the two raw source files in MySQL exactly as received, dirt included. Staging is the "undo button" for the pipeline: every later cleaning rule can be re-run or corrected because the original data is preserved here, untouched.

Nothing is cleaned, typed, deduplicated or standardized in this layer.

2. Inputs and outputs

Detail	
Input	<code>claims_raw.csv</code> (~1.01M rows, 35 columns), <code>membership_raw.csv</code> (~1.21M rows, 7 columns)
Source	Synthetic generator (<code>generate.py</code>), designed per <code>Data_Generation_Methodology.pdf</code> . Simulates a batch file drop from a claims system and an enrollment system
Output	<code>staging_claims_raw</code> , <code>staging_membership_raw</code> in database <code>claims_forecasting</code>

3. Grain (what one row means)

Table	One row =
<code>staging_claims_raw</code>	One claim record as it appeared in the file. Duplicates and adjustment records are separate rows
<code>staging_membership_raw</code>	One member in one month, as reported. Gaps and duplicates are preserved

4. Design decisions

Decision	Why
Every source column is <code>VARCHAR</code>	Staging must never reject or reinterpret dirty data. Blank numbers, malformed timestamps and mixed-case status text all load safely. Casting happens in the core layer, where we control what a bad value becomes
Added <code>row_id INT AUTO_INCREMENT PRIMARY KEY</code>	<code>claim_id</code> has deliberate duplicates, so it cannot be a key. <code>row_id</code> gives every raw row a unique identity for tracing and dedup
Added <code>_loaded_at TIMESTAMP</code>	Records when each row was loaded. Metadata about the load, not part of the source data
<code>DROP TABLE IF EXISTS</code> then <code>CREATE</code>	Makes the script safely rerunnable (idempotent). Rerunning gives a clean reload, never duplicated rows
Bulk <code>LOAD DATA INFILE</code> instead of the Workbench import wizard	The wizard inserts row by row (very slow at ~1M rows) and auto-guessed column types
Server-side load from the <code>secure_file_priv</code> folder	See section 6. This mirrors production: the server reads from a controlled location, not from an arbitrary client machine
Load column list excludes <code>row_id</code> and <code>_loaded_at</code>	MySQL fills these itself. The list must match the CSV's column order exactly
CSVs are not committed to Git	Source data does not belong in version control. <code>.gitignore</code> excludes <code>*.csv</code> and <code>data/</code> . The README explains how to regenerate

5. Schema notes to carry forward

- **Reserved word:** the CSV column is named `year_month`, but `YEAR_MONTH` is a reserved keyword in MySQL. The membership table therefore uses `month_year`. The load maps by position, so the data is unaffected. Anywhere we reference the CSV's `year_month`, the table column is `month_year`.
- `birth_date` is `VARCHAR(40)`: the source values include a time component, not just a date. Core will cast to `DATE`.
- All date, amount, flag and age columns are text at this layer and must be cast in core.

6. Issues encountered and how they were resolved

#	Symptom	Root cause	Resolution	Lesson
1	Import wizard failed at row 1366: incorrect integer value <code>('')</code> for <code>procedure_code</code>	Wizard auto-typed the column as integer. About 4.5% of procedure codes are blank by design	Abandoned the wizard for a scripted load with all <code>VARCHAR</code> columns	Never let a tool guess types on dirty data. This is why staging is untyped
2	Wizard import extremely slow	Row-by-row inserts	Switched to <code>LOAD DATA INFILE</code>	Use the database's bulk loader for large files
3	Error 1064 creating the membership table near <code>year_month</code>	<code>year_month</code> is a reserved word in MySQL	Renamed the column to <code>month_year</code>	Check identifiers against reserved words
4	Error 2068 on <code>LOAD DATA LOCAL INFILE</code>	Client-side restriction. Local file loading is disabled by default because it is a known security risk	Switched to server-side <code>LOAD DATA INFILE</code>	The blocked default is a real safeguard, not an obstacle
5	Error 1290 (<code>--secure-file-priv</code>)	File path did not match the folder MySQL allows	Put the CSVs in the folder returned by <code>SHOW VARIABLES LIKE 'secure_file_priv'</code> and used a forward-slash path	Server file access is restricted to one approved folder
6	Errors 1054 and 1366 on the membership load	A corrupted column list (pasted text merged, <code>row_id</code> included in the list) shifted every value one column	Rewrote the column list: exactly 7 names, matching CSV order, no auto-increment column	The load column list must mirror the file, not the table
7	Error 1406, data too long for <code>birth_date</code>	Values include a time component, longer than <code>VARCHAR(20)</code>	Widened to <code>VARCHAR(40)</code>	Real data surprises column widths.

#	Symptom	Root cause	Resolution	Lesson
				Widening in staging is a normal fix

7. Validation

Row counts after load (fill in from your own run):

Table	Expected (from generator)	Actual
staging_claims_raw	1,014,550	-----
staging_membership_raw	1,214,303	-----

Counts equal to the CSV data rows (file lines minus the header) mean nothing was dropped or duplicated during load. A mismatch means investigate before moving on.

8. What staging deliberately does not do

- No casting to numeric or date types
- No deduplication (duplicate `claim_id` rows stay)
- No standardization (region spellings and `claim_status` variants stay)
- No orphan handling (claims whose `member_id` is missing from membership stay)
- No adjustments applied

All of the above is decided from evidence in the profiling step and implemented in the core layer.

9. Interview one-liners

- **30 sec:** "I land raw files in an untyped staging layer so nothing is lost or silently altered, then clean in later layers. It's the undo button for the pipeline."
- **Why VARCHAR everywhere:** "The import tool guessed an integer type for a column with blanks and failed. Staging shouldn't guess. I cast deliberately in the next layer."
- **Why server-side load:** "Client-side local loading is disabled by default because it's a security risk. Production loads read from a controlled location, so I did the same."